

Geometric Background

ROBOTICS — Lecture 03

Matlab Implementation of Rotations

Fundamental Rotation Matrices

The rotation matrix about the z -axis is:

$$R_{z(\alpha)} = \begin{pmatrix} \cos \alpha & -\sin \alpha & 0 \\ \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Matlab implementation `zRot`:

% Function that returns the Rotation about z axis

```
function [Rz] = zRot(alpha)
    Rz = [
        cos(alpha), -sin(alpha), 0;
        sin(alpha),  cos(alpha), 0;
        0,          0,          1
    ];
end
```

Rotation about the y -axis (`yRot(beta)`):

$$R_{y(\beta)} = \begin{pmatrix} \cos \beta & 0 & \sin \beta \\ 0 & 1 & 0 \\ -\sin \beta & 0 & \cos \beta \end{pmatrix}$$

Rotation about the x -axis (`xRot(gamma)`):

$$R_{x(\gamma)} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \gamma & -\sin \gamma \\ 0 & \sin \gamma & \cos \gamma \end{pmatrix}$$

The same can be done for the other fundamental rotation matrices.

Drawing a Reference Frame

`scriptDraw` builds a 3D plot (`close all, view(3), daspect([1 1 1]), hold on`) and draws the three axes with `plot3` in red, green, blue. Then it rotates the three unit vectors by 45 degrees about z : `v1 = zRot(pi/4)*[1;0;0]`, `v2 = zRot(pi/4)*[0;1;0]`, `v3 = zRot(pi/4)*[0;0;1]`, and plots the rotated vectors dashed.

Still on Rotation Matrices

Composition of Rotation Matrices

Example with three frames sharing the same origin: $O - x_0y_0z_0$, $O - x_1y_1z_1$, $O - x_2y_2z_2$. A point P has coordinates p_0, p_1, p_2 in the three frames. Let R_{ij} denote the rotation matrix of Frame i with respect to Frame j . Then:

$$p_1 = R_{21}p_2 \quad p_0 = R_{10}p_1 \quad p_0 = R_{20}p_2$$

By substitution and comparison:

$$R_{20} = R_{10}R_{21}$$

The relation can be interpreted in two steps:

1. First, rotate $O - x_0y_0z_0$ so as to align it with frame $O - x_1y_1z_1$ (matrix R_{10}).
2. Then rotate again so as to align frame $O - x_1y_1z_1$ with $O - x_2y_2z_2$ (matrix R_{21}).

Composition of Rotations A rotation can be expressed as a sequence of partial rotations, each one defined with respect to the previous one. The frame from which the rotation occurs is termed the **current frame**. The sequence is obtained by post-multiplying the matrices associated with each rotation.

In view of the properties of rotation matrices:

$$R_{ij} = (R_{ji})^{-1} = (R_{ji})^T$$

Example: rotations about current axes

Sequence about axes in the current frame:

1. First rotate by φ about the current y -axis.
2. Then rotate by θ about the current z -axis.

Result:

$$R_{20} = R_{y(\varphi)}R_{z(\theta)}$$

Another example:

$$R_{40} = R_{z(\alpha)}R_{y(\beta)}R_{y(\gamma)}R_{z(\delta)}$$

The order is absolutely important:

$$R_{z(\alpha)}R_{y(\beta)}R_{y(\gamma)}R_{z(\delta)} \neq R_{y(\gamma)}R_{z(\delta)}R_{z(\alpha)}R_{y(\beta)}$$

Rotation about Fixed Axes

To compose rotations about fixed axes, consider two rotated frames $O - x_0y_0z_0$ and $O - x_1y_1z_1$. For a generic point P : $p_0 = R_{10}p_1$. Consider a generic linear application A defined in the $O - x_0y_0z_0$ frame. For a generic point P and its transformed point:

$$q_0 = Ap_0 \quad q_1 = R_{01}Ap_0 = R_{01}AR_{10}p_1 = (R_{10})^{-1}AR_{10}p_1$$

Example: the first rotation is $R_{10} = R_{y(\varphi)}$ about the y_0 -axis. The rotation R_2^1 happens about the z_0 -axis (not the z_1 -axis, as in the current-frame case):

$$R_{20} = R_{10}R_2^1$$

Since the latter is known in the current axes:

$$R_2^1 = (R_{10})^{-1}R_{z(\theta)}R_{10}$$

which leads to:

$$R_{20} = R_{10}(R_{10})^{-1}R_{z(\theta)}R_{10} = R_{z(\theta)}R_{y(\varphi)}$$

Rule When the composition of rotations happens with fixed axes, we have to **pre-multiply** instead of post-multiplying (as is the case for rotations in the current frame). Example:

$$R_{40} = R_{z(\delta)}R_{y(\gamma)}R_{y(\beta)}R_{z(\alpha)}$$

Rotation about Current Axes — Example

Consider the rotation composed as follows:

1. A rotation of θ about the current x -axis.
2. A rotation of φ around the current z -axis.

Result:

$$R_{20} = R_{x(\theta)}R_{z(\varphi)}$$

A point expressed as p_2 in $O - x_2y_2z_2$ corresponds to $p_0 = R_{20}p_2$ in the initial frame.

A more complex example

1. A rotation of θ about the current x -axis.
2. A rotation of φ around the current z -axis.
3. A rotation of α about the fixed z_0 -axis.

Following the same procedure:

$$R_{30} = R_{10}R_{21}R_3^2 = R_{x(\theta)}R_{z(\varphi)}R_3^2$$

where R_3^2 denotes the rotation with respect to the initial fixed axes. The effect of the rotation about the fixed frame in the current frame $O - x_2y_2z_2$ is:

$$R_3^2 = (R_{10}R_{21})^{-1}R_{z(\alpha)}(R_{10}R_{21}) = (R_{x(\theta)}R_{z(\varphi)})^{-1}R_{z(\alpha)}R_{x(\theta)}R_{z(\varphi)}$$

which leads to:

$$R_{30} = R_{z(\alpha)}R_{x(\theta)}R_{z(\varphi)}$$

The Final Example Composition: (1) θ about the current x -axis; (2) φ around the current z -axis; (3) α about the fixed z_0 -axis; (4) β around the current y -axis; (5) δ about the fixed x_0 -axis.

$$R_{r0} = R_{x(\delta)}R_{z(\alpha)}R_{x(\theta)}R_{z(\varphi)}R_{y(\beta)}$$

Parametrisation of Rotation Matrices

1. A 3D rotation matrix has 9 elements, but a rotation is completely specified by three parameters.
2. A rotation is an element of $SO(m)$ completely specified by $\frac{m(m-1)}{2}$ parameters: 1 parameter for $SO(2)$, 3 parameters for $SO(3)$.
3. A 3D rotation is completely specified by three rotations about different axes, such that no two adjacent rotations are about the same axis.

Count Choice

3x	Any of the three axes is acceptable for the first rotation
2x	Rule out the axis used for the first rotation
2 =	Rule out the axis used for the second rotation
12	Total parametrisations with three rotations

Euler Angles

1. Euler angles are the angles related to three rotations about the current axis.
2. A very popular convention is the use of XYZ rotations about fixed axes, known in the aeronautics world as **roll-pitch-yaw**, because these rotations denote the change in attitude of an aircraft with respect to three fixed axes.

3. Since the rotation is about fixed axes, we have to pre-multiply:

$$R = R_{z(\varphi)} R_{y(\theta)} R_{x(\psi)}$$

4. ψ is the roll angle, θ is the pitch angle, φ is the yaw angle.

Euler — Matlab

Any rotation matrix can be created by specifying three angles, e.g. the triple of Euler angles (three rotations about the current axes: ZYZ):

$$R(\varphi, \theta, \psi) = R_{z(\varphi)} R_{y'(\theta)} R_{z''(\psi)}$$

```
function [R] = eulerRot(phi, theta, psi)
    R = zRot(phi) * yRot(theta) * zRot(psi);
end
```

Tested by scriptDrawEuler: $R = \text{eulerRot}(\pi/6, \pi/6, 0)$, then $v1 = R*[1;0;0]$, $v2 = R*[0;1;0]$, $v3 = R*[0;0;1]$, plotted dashed.

RPY Angles

Introduce the notation $c_\alpha = \cos(\alpha)$, $s_\alpha = \sin(\alpha)$. Then:

$$R = \begin{pmatrix} c_\varphi c_\theta & c_\varphi s_\theta s_\psi - s_\varphi c_\psi & c_\varphi s_\theta c_\psi + s_\varphi s_\psi \\ s_\varphi c_\theta & s_\varphi s_\theta s_\psi + c_\varphi c_\psi & s_\varphi s_\theta c_\psi - c_\varphi s_\psi \\ -s_\theta & c_\theta s_\psi & c_\theta c_\psi \end{pmatrix}$$

Given a rotation matrix $R = \begin{pmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{pmatrix}$, the inverse problem has

two solutions:

$$\theta \in \left(-\frac{\pi}{2}, \frac{\pi}{2}\right) \rightarrow \begin{cases} \varphi = \text{atan2}(r_{21}, r_{11}) \\ \theta = \text{atan2}(-r_{31}, \sqrt{r_{32}^2 + r_{33}^2}) \\ \psi = \text{atan2}(r_{32}, r_{33}) \end{cases}$$

$$\theta \in \left(\frac{\pi}{2}, 3\frac{\pi}{2}\right) \rightarrow \begin{cases} \varphi = \text{atan2}(-r_{21}, -r_{11}) \\ \theta = \text{atan2}(-r_{31}, -\sqrt{r_{32}^2 + r_{33}^2}) \\ \psi = \text{atan2}(-r_{32}, -r_{33}) \end{cases}$$

Gimbal Lock

1. An important feature of RPY, and generally of Euler-based representations, is that the three angles act as three independent degrees of freedom.
2. At a pitch angle of $\pm\frac{\pi}{2}$, the roll and yaw rotation axes become aligned. This results in the loss of a degree of freedom.
3. A rotation around one axis can be replicated by a rotation around the other. Indeed, when $\theta = \frac{\pi}{2}$:

$$R = \begin{pmatrix} 0 & s(\varphi + \psi) & c(\varphi + \psi) \\ 0 & c(\varphi + \psi) & -s(\varphi + \psi) \\ -1 & 0 & 0 \end{pmatrix}$$

Here φ and ψ do not act independently: all rotations achieved with a combination of φ and ψ can be accomplished by a single rotation of angle $(\varphi + \psi)$.

1. The inverse mapping from the rotation matrix to the RPY angles is singular at this point: infinitely many combinations of φ and ψ produce the same rotation matrix.
2. This problem has profound reasons: it is like creating a 2D map from a sphere. When you flatten it, the poles become lines; a single point on the sphere is mapped to an infinite number of points.

Consequence This lack of invertibility can cause a significant problem: in the singular configuration, small changes in the physical orientation can produce discontinuous jumps in the calculated RPY angles. This is a major issue for physical systems like robots, which require smooth and continuous control.

Axis–Angle Representation

1. Adopt a redundant parametrisation using an axis ($k = [k_x, k_y, k_z]$ with $|k| = 1$) and an angle (θ), resulting in four parameters overall.
2. The rotation $R_{k(\theta)}$ can be found by transforming the coordinates. Find a new frame $O - x_1 y_1 z_1$, with z_1 coincident with k . Rotate $O - x_0 y_0 z_0$ by α about the z -axis and by β around the y -axis.
3. Clearly $R_{10} = R_{z(\alpha)} R_{y(\beta)}$. If a transformation A (e.g. a rotation) is expressed in $O - x_1 y_1 z_1$, as in $q_1 = A p_1$, then:

$$q_0 = (R_{01})^{-1} A R_{01} p_0 = R_{z(\alpha)} R_{y(\beta)} A (R_{z(\alpha)} R_{y(\beta)})^{-1}$$

which leads to:

$$R_{k(\theta)} = R_{z(\alpha)} R_{y(\beta)} R_{z(\theta)} R_{y(-\beta)} R_{z(-\alpha)}$$

Using trigonometry:

$$\sin(\alpha) = \frac{k_y}{\sqrt{k_x^2 + k_y^2}} \quad \cos(\alpha) = \frac{k_x}{\sqrt{k_x^2 + k_y^2}}$$

$$\sin(\beta) = \frac{\sqrt{k_x^2 + k_y^2}}{k_z} \quad \cos(\beta) = k_z$$

Rodrigues' formula

$$R_{k(\theta)} = \begin{pmatrix} k_x^2(1 - c_\theta) + c_\theta & k_x k_y(1 - c_\theta) - k_z s_\theta & k_x k_z(1 - c_\theta) + k_y s_\theta \\ k_x k_y(1 - c_\theta) + k_z s_\theta & k_y^2(1 - c_\theta) + c_\theta & k_y k_z(1 - c_\theta) - k_x s_\theta \\ k_x k_z(1 - c_\theta) - k_y s_\theta & k_y k_z(1 - c_\theta) + k_x s_\theta & k_z^2(1 - c_\theta) + c_\theta \end{pmatrix}$$

1. Four parameters (k, θ) plus the constraint $k_x^2 + k_y^2 + k_z^2 = 1$: the system has three degrees of freedom.
2. From the matrix: $R_{-k}(-\theta) = R_{k(\theta)}$; inverting both the axis and the angle gives the same rotation.
3. The axis–angle parametrisation is not globally invertible, but it is locally invertible (to be precise, a local homeomorphism). Unlike RPY angles, there are no singularities such as Gimbal Lock.
4. The problem, compared to RPY angles, is that this parametrisation is not very intuitive.
5. Unlike using rotation matrices directly, it is not straightforward to analytically compose or invert subsequent rotations.

$$\text{Frame } O - x_0 y_0 z_0 \rightarrow R_{z(\alpha)} R_{y(\beta)}: z_1 \Rightarrow k \rightarrow \text{rotate } \theta \text{ about } z_1 = k$$

Fig. 1: Axis–angle construction: frame rotated so that z_1 coincides with k , then rotated by θ about k .

Quaternions

Fundamentals

1. For a real number $x \in \mathbb{R}$ the inverse is $x^{-1} = \frac{1}{x}$.
2. For an element of $\mathbb{R}^2$, $x = \begin{pmatrix} a \\ b \end{pmatrix}$, an inverse exists using complex numbers: $z = a + ib$, with i the imaginary unit. Introducing the complex conjugate $z^* = a - ib$ and $z_1 = \frac{z^* z}{|z|^2}$: $z \cdot z_1 = \frac{z z^* z}{a^2 + b^2} = \frac{a^2 + b^2}{a^2 + b^2} = 1$, hence $z^{-1} = \frac{z^*}{|z|^2}$.
3. The same trick cannot be repeated with $\mathbb{R}^3$, but it can with $\mathbb{R}^4$, by using **quaternions**.

Definition of a Complex Number

1. Let 1 and i be the basis elements.
1. The following axiom holds: $i^2 = -1$.
2. A complex number is an element of a vector space over the reals, with the form $z = x + iy$, where $x, y \in \mathbb{R}$.

Definition of a Quaternion

1. Let 1 and i, j, k be the basis elements.
1. The following axioms hold: $i^2 = j^2 = k^2 = ijk = -1$.
2. A quaternion is an element of a vector space over the reals, with the form $q = q_0 + q_1 i + q_2 j + q_3 k$.
3. Shorthand distinguishes the **scalar part** q_0 and the **vector part** $q_{\cdot} = q_1 i + q_2 j + q_3 k$: $q = q_0 + q_{\cdot}$. An equivalent notation is $[q_0, q_{\cdot}]$.

Background on vector products

Scalar (dot) product. Given $u = u_1 i + u_2 j + u_3 k$ and $v = v_1 i + v_2 j + v_3 k$, the scalar product is the real number $u \cdot v = u_1 v_1 + u_2 v_2 + u_3 v_3$. If θ is the angle between the vectors, $u \cdot v = |u| |v| \cos \theta$. It represents the projection of one vector onto the other and is commutative.

Vector (cross) product. The vector product $s = u \times v$ is a vector orthogonal to both u and v , whose magnitude is $|s| = |u| |v| \sin \theta$.

$s = u \times v = s_1 i + s_2 j + s_3 k$ with $\begin{cases} s_1 = u_2 v_3 - u_3 v_2 \\ s_2 = u_3 v_1 - u_1 v_3 \\ s_3 = u_1 v_2 - u_2 v_1 \end{cases}$. The vector product

is anti-commutative: $u \times v = -v \times u$.

Quaternion Product, Conjugate and Inverse

From the axiom $i^2 = j^2 = k^2 = ijk = -1$, the multiplication rules of the basis elements are:

$$ij = k, ji = -k; \quad jk = i, kj = -i; \quad ki = j, ik = -j$$

These rules are the same as those for the vector cross product.

With the shorthand $p = p_0 + p_{\cdot}$ and $q = q_0 + q_{\cdot}$:

$$pq = (p_0 + p_{\cdot})(q_0 + q_{\cdot}) = p_0 q_0 + p_0 q_{\cdot} + q_0 p_{\cdot} + p_{\cdot} q_{\cdot}$$

The product of the two vector parts simplifies as $p_{\cdot} q_{\cdot} = -p_{\cdot} \cdot q_{\cdot} + p_{\cdot} \times q_{\cdot}$. Thus the full product is:

$$pq = (p_0 q_0 - p_{\cdot} \cdot q_{\cdot}) + (p_0 q_{\cdot} + q_0 p_{\cdot} + p_{\cdot} \times q_{\cdot})$$

Scalar part: $p_0 q_0 - p_{\cdot} \cdot q_{\cdot}$; vector part: $p_0 q_{\cdot} + q_0 p_{\cdot} + p_{\cdot} \times q_{\cdot}$. The quaternion product is **associative** but **not commutative** ($pq \neq qp$).

Quaternion Conjugate

$$q = q_0 + q_1 i + q_2 j + q_3 k \implies q^* = q_0 - q_1 i - q_2 j - q_3 k$$

$$qq^* = (q_0 + q_{\cdot})(q_0 - q_{\cdot}) = q_0^2 - q_{\cdot}^2 = q_0^2 + |q_{\cdot}|^2 = q_0^2 + q_1^2 + q_2^2 + q_3^2$$

Quaternion Length

$$|q| = \sqrt{qq^*} = \sqrt{q_0^2 + q_1^2 + q_2^2 + q_3^2}$$

is the quaternion length, or norm.

A quaternion (except 0) has a unique inverse:

$$q^{-1} = \frac{q^*}{|q|^2} \quad \text{with} \quad \frac{qq^*}{|q|^2} = 1$$

The relation simplifies further when the quaternion has unit length.

To complex numbers and back

A complex number can be expressed in Euler form:

$$z = \rho(\cos(\theta) + j \sin(\theta))$$

Multiplying two complex numbers in Euler notation:

$$z_1 z_2 = \rho_1 \rho_2 (\cos(\theta_1 + \theta_2) + j \sin(\theta_1 + \theta_2))$$

Multiplying a complex number by a unit-norm (unit-modulus) complex number has the effect of a rotation. Multiplying by its complex conjugate “undoes” the rotation, aligning the result with the real axis.

Quaternions and Rotations

Consider a unit-length quaternion $q = [\eta, \varepsilon]$, where η is the scalar part and $\varepsilon = \begin{pmatrix} \varepsilon_x \\ \varepsilon_y \\ \varepsilon_z \end{pmatrix}$ the vector part. It can be expressed as:

$$q = \left(\cos\left(\frac{\theta}{2}\right), \sin\left(\frac{\theta}{2}\right)k \right) \quad \text{where} \quad k = \frac{\varepsilon}{|\varepsilon|} \quad \theta = 2 \operatorname{atan2}(|\varepsilon|, \eta)$$

Key Fact The unit-length quaternion $q = (\cos(\frac{\theta}{2}), \sin(\frac{\theta}{2})k)$ represents a rotation of angle θ about the axis defined by the unit vector k .

Let $q = [\eta, \varepsilon]$ be a unit quaternion and $p = [0, p.]$ a pure quaternion representing a vector p . The rotated vector p' is computed by the conjugation formula

$$p' = qpq^*$$

, yielding a pure quaternion $[0, p']$. After computation, the vector part p' is:

$$p' = (\eta^2 - |\varepsilon|^2)p + 2\eta(\varepsilon \times p) + 2(\varepsilon \cdot p)\varepsilon$$

By substituting $\eta = \cos(\frac{\theta}{2})$, $\varepsilon = \sin(\frac{\theta}{2})k$ and $|\varepsilon|^2 = \sin^2(\frac{\theta}{2})$:

$$p' = \cos(\theta)p + \sin(\theta)(k \times p) + (1 - \cos(\theta))(k \cdot p)k$$

Rodrigues' rotation formula

$$p' = \cos(\theta)p + \sin(\theta)(k \times p) + (1 - \cos(\theta))(k \cdot p)k$$

It describes the rotation of a vector p by an angle θ about the axis k .

Equivalent rotation matrix

Rodrigues' formula is equivalent to applying a rotation matrix $R_{k(\theta)}$:

$$p' = R_{k(\theta)}p$$

, where:

$$R_{k(\theta)} = \begin{pmatrix} k_x^2(1 - c_\theta) + c_\theta & k_x k_y(1 - c_\theta) - k_z s_\theta & k_x k_z(1 - c_\theta) + k_y s_\theta \\ k_x k_y(1 - c_\theta) + k_z s_\theta & k_y^2(1 - c_\theta) + c_\theta & k_y k_z(1 - c_\theta) - k_x s_\theta \\ k_x k_z(1 - c_\theta) - k_y s_\theta & k_y k_z(1 - c_\theta) + k_x s_\theta & k_z^2(1 - c_\theta) + c_\theta \end{pmatrix}$$

with $c_\theta = \cos(\theta)$ and $s_\theta = \sin(\theta)$.

Summary Given a unit-length quaternion $q = [\eta, \varepsilon] = (\cos(\frac{\theta}{2}), \sin(\frac{\theta}{2})k)$, associate it with a rotation of angle θ about the unit vector k , with $k = \frac{\varepsilon}{|\varepsilon|}$ and $\theta = 2 \operatorname{atan2}(|\varepsilon|, \eta)$. The parameters correspond to three degrees of freedom:

1. four numbers $\eta, \varepsilon_x, \varepsilon_y, \varepsilon_z$ with constraint $\eta^2 + \varepsilon_x^2 + \varepsilon_y^2 + \varepsilon_z^2 = 1$;
1. four numbers θ, k_x, k_y, k_z with constraint $k_x^2 + k_y^2 + k_z^2 = 1$.

From Rotation Matrix to Quaternions

Given a rotation matrix $R = \begin{pmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{pmatrix}$, the inverse problem is solved by:

$$\eta = \frac{1}{2} \sqrt{r_{11} + r_{22} + r_{33} + 1}$$

$$\varepsilon = \frac{1}{2} \begin{pmatrix} \operatorname{sgn}(r_{32} - r_{23}) \sqrt{r_{11} - r_{22} - r_{33} + 1} \\ \operatorname{sgn}(r_{13} - r_{31}) \sqrt{r_{22} - r_{33} - r_{11} + 1} \\ \operatorname{sgn}(r_{21} - r_{12}) \sqrt{r_{33} - r_{11} - r_{22} + 1} \end{pmatrix}$$

$$\text{where } \operatorname{sgn}(x) = \begin{cases} 1 & x \geq 0 \\ -1 & x < 0 \end{cases}$$

Composition and Advantages

Consider two rotations associated with unit-length quaternions q_0 and q_1 . The first transforms p into p' , the second p' into p'' . Using pure quaternions $p = [0, p.]$, $p' = [0, p'.]$, $p'' = [0, p''.]$:

$$p'' = q_1 p' q_1^* = q_1 (q_0 p q_0^*) q_1^* = (q_1 q_0) p (q_1 q_0)^*$$

The last step holds because the conjugate of a product is the product of the conjugates in reverse order, $(q_1 q_0)^* = q_0^* q_1^*$. The composition of two rotations is therefore equivalent to the product of their quaternions.

Operation	Rotation Matrices	Unit-Quaternions
Inversion	$R^{-1} = R^T$	$q^{-1} = (\eta, -\varepsilon)$
Composition	$R = R_1 R_2$	$q = q_1 q_2$

Composition formula:

$$q_1 q_2 = (\eta_1 \eta_2 - \varepsilon_1 \cdot \varepsilon_2, \eta_1 \varepsilon_2 + \eta_2 \varepsilon_1 + \varepsilon_1 \times \varepsilon_2)$$

1. **Inversion:** the inverse of a unit-length quaternion is its conjugate, found by negating the vector part. This is computationally much faster than transposing and inverting a matrix.
2. **Composition:** the composition of two rotations is a simple quaternion product; the result represents the combined rotation. More efficient and numerically stable than composing rotation matrices.

In summary

1. **Compact representation:** four numbers instead of the nine of a rotation matrix.
2. **Absence of Gimbal Lock:** unlike Euler angles, quaternions provide a continuous and non-singular representation of rotations, completely avoiding the Gimbal Lock problem. Significant advantage in computer graphics and robotics.
3. **Efficient composition:** achieved by a simple quaternion multiplication, computationally more efficient than matrix multiplication.
4. **Direct interpolation:** quaternions allow smooth, constant-velocity interpolation between two rotations (spherical linear interpolation, *slerp*), crucial for animations.
5. **Non-uniqueness:** a single rotation can be represented by two quaternions, q and $-q$; a minor redundancy that does not demean the practical benefits.

Homogeneous Transformations**Position of a Frame in 3D**

A rigid body's pose is defined by the position and orientation of a frame attached to it. So far only rotation was considered, ignoring translation (with coincident frame origins). In the general case, both rotation and translation are present. A point p^1 in frame $\{1\}$ can be expressed in frame $\{0\}$ as:

$$p^0 = o_1^0 + R_1^0 p^1$$

Direct and Inverted Coordinate Transformation

The inverse relation is found by isolating p^1 :

$$p^1 = (R_1^0)^{-1} (p^0 - o_1^0) = R_1^{0T} (p^0 - o_1^0) = R_1^0 p^0 - R_1^0 o_1^0$$

This representation is rather cumbersome, as the inverse involves both a matrix transpose and a matrix-vector product.

Homogeneous Coordinates

A compact way to represent both rotation and translation: create the homogeneous representation $\tilde{p}$ of p by appending a coordinate of value 1:

$$\tilde{p} = \begin{pmatrix} p \\ 1 \end{pmatrix}$$

Construct a 4×4 matrix, the homogeneous transformation matrix:

$$A_1^0 = \begin{pmatrix} R_1^0 & o_1^0 \\ 0^T & 1 \end{pmatrix}$$

The transformation of a point from frame $\{1\}$ to frame $\{0\}$ becomes a single matrix-vector multiplication:

$$\tilde{p}^0 = A_1^0 \tilde{p}^1$$

A key advantage is that the coordinate transformation is a simple matrix multiplication. The inverse transformation is found by simply inverting the matrix:

$$\tilde{p}^1 = A_1^0 \tilde{p}^0 = (A_1^0)^{-1} \tilde{p}^0$$

The inverse homogeneous transformation matrix is:

$$A_0^1 = (A_1^0)^{-1} = \begin{pmatrix} R_1^{0T} & -R_1^{0T} o_1^0 \\ 0^T & 1 \end{pmatrix}$$

1. Unlike rotation matrices, homogeneous transformations are not orthogonal: for a general homogeneous matrix A , $A^{-1} \neq A^T$.
2. The formalism generalizes the properties of the rotation group to include translations. A sequence of transformations in the current axes is a simple matrix product:

$$\tilde{p}^0 = A_1^0 A_2^1 \dots A_n^{n-1} \tilde{p}^n$$
3. The set of all homogeneous transformations forms a group, the **special Euclidean group** $SE(3) = \mathbb{R}^3 \rtimes SO(3)$ (semi-direct product).

Matlab Examples for Homogeneous Transformations

`homogeneousTrans` builds a homogeneous transformation:

```
% homogeneous transformation
```

```
function [T] = homogeneousTrans(phi, theta, psi, Ov)
```

```
    R = eulerRot(phi, theta, psi);
```

```
    T = [R, Ov;
```

```
        zeros(1,3), 1]
```

```
end
```

$$A_1^0 = \begin{pmatrix} R(\varphi, \theta, \psi) & O_1^0 \\ O & 1 \end{pmatrix}$$

1. `scriptDrawHom`: $T = \text{homogeneousTrans}(0, 0, \pi/4, [1;1;1])$; $O1 = T*[0;0;0;1]$; $v1 = T*[1;0;0;1]$, $v2 = T*[0;1;0;1]$, $v3 = T*[0;0;1;1]$; the transformed frame is plotted dashed.
2. `scriptDrawHom1`: equivalent using built-in Matlab functions: $T = \text{makehgtform}(\text{'translate'}, [1,1,1], \text{'zrotate'}, \pi/4)$.
3. `scriptTestTriad`: uses a downloaded community file to draw axis frames: $h = \text{triad}(\text{'Parent'}, \text{axis})$; then $h1 = \text{triad}(\text{'Parent'}, h, \text{'Matrix'}, \text{makehgtform}(\text{'translate'}, [1,1,1], \text{'xrotate'}, \pi/4), \text{'linewidth'}, 3, \text{'linestyle'}, \text{'--'})$.
4. `scriptTransformTest`: Matlab provides an object to create a transformation, link it to the current axis, and define child objects transformed accordingly: $t = \text{hgtransform}(\text{'Parent'}, \text{axis})$; a cylinder is created with `cylinder([1,1],100)` and `surface`; `set(h, 'Parent', t)` associates it; then $Rz = \text{makehgtform}(\text{'xrotate'}, 2*\pi/3)$, `set(t, 'Matrix', Rz)`, and `drawnow` updates the children.
5. `scriptTransformTriads`: combines triads and `hgtransform`; a second frame is obtained with `makehgtform('translate', [1,1,1], 'xrotate', pi/4)`, then a loop `for angle = 0:0.5:360` applies $Rz = \text{makehgtform}(\text{'zrotate'}, \text{deg2rad}(\text{angle}))$ and `drawnow` animates it.

$$\boxed{\text{Frame } \{0\}} \rightarrow \boxed{p^0 = o_1^0 + R_1^0 p^1} \rightarrow \boxed{\tilde{p}^0 = A_1^0 \tilde{p}^1}$$

Fig. 2: Rigid transformation: translation plus rotation, compactly written with homogeneous coordinates.